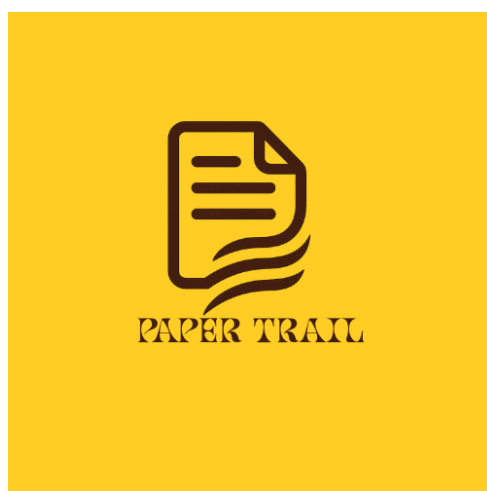




PaperTrail

Administrator Manual

Dr. Zihan Wu

Ben Yandell, Robert Kulow, Anthony Veilleux, Arius Ahmad, Vasu Patel

03/25/2026

V1.0

Table of Contents

1. Introduction.....	3
1.1 Purpose of This Document.....	3
References.....	4
2. System Overview.....	4
2.1 Background.....	4
2.2 Hardware and Software Requirements.....	5
3. Administrative Procedures.....	5
3.1 Installation.....	5
When fully released:.....	6
To install the dev build:.....	6
Requirements.....	6
Steps.....	6
1. Install clasp (Google Apps Script CLI).....	6
2. Clone the project.....	6
3. Log into Google.....	6
4. Create an Apps Script project.....	7
5. Get the latest code.....	7
6. Push code to your Apps Script project.....	7
7. Open the project.....	7
3.2 Routine Tasks.....	7
3.3 Periodic Administration.....	7
3.4 User Support.....	8
4. Troubleshooting.....	8
4.1 Dealing with Error Messages and Failures.....	8
4.2 Known Bugs and Limitations.....	9
Appendix A – Team Review Sign-off.....	10
Appendix B – Document Contributions.....	11

1. Introduction

1.1 Purpose of This Document

This document is intended to outline the system requirements, minimum specifications, installation procedures, troubleshooting tactics, and a compilation of known bugs. By the end of this document, administrators should be comfortable operating and installing the software onto devices. As well as having knowledge of potential pitfalls that might hinder the user experience; how to circumvent them in the event of possible failures. The PaperTrail software is meant to be a lightweight and intuitive software that merely assists users; it shouldn't cause much overhead to confuse users.

Background information will be given to highlight what tasks administrators will need to do for day-to-day maintenance for the software; what kinds of hardware and software is expected to run said software. Administrative procedures are expanded to teach the installation process, routine tasks to keep the software operational, periodic administration activities to verify the availability and integrity of the software, and user support tactics in the event of further questions. Troubleshooting methods will be provided via a list of error codes and what to do in the event of such occurrence. Ending with a compilation of known bugs and limitations at the time of the document creation. Which will cause the software to dampen the user experience and add overhead for administrators.

References

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Requirements Specification*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Design Document*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail User Interface Design Document*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail Critical Design Review Document*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2026) *PaperTrail Code Inspection Report*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2026) *PaperTrail Administrator Manual*

GitHub Repository: <https://github.com/AnthonyVeilleux/PaperTrail>

2. System Overview

The following section will explain the administrative process needed to deploy the software. While also elaborating on the minimum hardware and software requirements needed for the software to run operationally.

2.1 Background

The system administrator for PaperTrail is responsible for the overall health, deployment, and general lifecycle of this Google-Workspace add-on. Their primary duties begin with installation, which involves pulling the latest code from the central GitHub repository and deploying it to production by the way of Google Clasp. After initial deployment, the administrator manages the daily operations by monitoring user reports, running tests and reviewing runtime errors. For ongoing maintenance and recovery, the system administrator relies on GitHub, which serves as the primary backup and source of truth. In the event of a catastrophic failure or erroneous deployment the administrator is tasked with rolling back the version to a previous stable commit in git and redeploying via clasp.

2.2 Hardware and Software Requirements

Minimum Hardware Requirements:

1. Computer/laptop capable of running a web browser
2. Minimum 4GB RAM
3. 1GB of local storage for source code, [Node.js](#), Git repository and other tools. (For developing)

Required Software:

1. Supported web browser
 - a. Google Chrome
 - b. Microsoft Edge
 - c. Safari
 - d. Mozilla Firefox
2. A valid Google Account
3. Access to Google Docs
4. Access to Google Drive
5. Permission to install add-ons
 - a. Allow the software to access read and write data to your Google Drive
6. Development Tools
 - a. Git
 - b. [Node.js](#) w/ npm

- c. Google Apps Script CLI (clasp)
- d. Access to AnthonyVeilleux/PaperTrail repo

3. Administrative Procedures

Administrative Procedures will go over how to install and manage the software. While providing basic user support by contacting the development team.

3.1 Installation

In a Google Doc:

- 1) Click Extensions Tab
- 2) Click Get Addons
- 3) In the search bar, search PaperTrail
- 4) Click on PaperTrail, and then Click Install.

To install the dev build:

Requirements

- Git
 - Node.js (npm included)
 - Google account
-

Steps

1. Install clasp (Google Apps Script CLI)

Run:

```
npm install -g @google/clasp
```

2. Clone the project

```
git clone https://github.com/AnthonyVeilleux/PaperTrail.git
```

```
cd PaperTrail
```

3. Log into Google

```
clasp login
```

A browser window will open—sign in with your Google account.

4. Create an Apps Script project

Option A (recommended):

```
clasp create --type standalone --title "Dev Environment"
```

Option B (manual):

1. Go to script.google.com
2. Create a new project
3. Copy the Script ID from the URL
4. Run:

```
clasp clone YOUR_SCRIPT_ID
```

5. Get the latest code

```
git pull
```

6. Push code to your Apps Script project

```
clasp push
```

7. Open the project

clasp open

3.2 Routine Tasks

Periodically the user must update the local database if they want to make use of the global search function. Without, the search query will never have up to date information about where tags are located across documents. Only local tags will be available to search query.

3.3 Periodic Administration

Periodic administration for PaperTrail is limited to maintaining the codebase integrity and ensuring compliance with Google's platform updates.

The source code is version-controlled in the GitHub repository, which serves as the permanent record and backup. There is no other data that needs to be pruned or cleaned as all of the data is maintained within a google document.

The administrator should periodically check for deprecation notices from Google Apps Script by checking the manifest version of the plugin on the workplace store, ensuring the code remains compatible with the latest platform version.

3.4 User Support

There is no planned live support for this product. However, for any queries regarding the software that its documentation cannot answer, below is a list of the contact information of the development team:

Ben Yandell – benjamin.yandell@maine.edu
Robert Kulow – robert.kulow@maine.edu
Anthony Veilleux – anthony.veilleux@maine.edu
Arius Ahmad – arius.ahmad@maine.edu
Vasu Patel – vasu.patel@maine.edu

4. Troubleshooting

This section will outline the procedures to follow when errors inevitably occur. Educating admins and users how to recover the software with the least damage possible.

4.1 Dealing with Error Messages and Failures

Common errors that are known to occur in normal use:

Error	Context	Probable Cause
“Tag is required”	Export Dialog	The user tried to export without selecting a tag. Select a tag first.
“No notes found for #TagName”	Export Service	Tag exists, but no matching paragraphs were found in the document.
"Error refreshing tags: ...	Menu Action	Failure in <code>extractHashtagsFromDocument</code> . Check execution logs for details.
“Script is missing OAuth scopes”	Authorization	Manifest changed but not re-authorized. Reload the sidebar and authorize again. Also be sure to be logged into the correct account

For helping a user with an error:

To replicate the error, ask the user for the action they took. Make sure that they are meeting the requirements for that specific action. If needed, ask the user for their document URL to verify that everything is set up correctly.

Then check the logs of the script by going inside of the appscripts project and then clicking Executions.

4.2 Known Bugs and Limitations

Bugs

The most critical bug is in `renameTagInDocument`, which uses `"tagName"` in the regex pattern instead of the `"oldName"` parameter that was passed in which means tag renames may search for the wrong tag entirely. In `getTagOccurrences`, the variable `body` is redeclared inside an if-statement using `var body`, which can cause unexpected scoping behavior. In `buildTagBookmarks`, if `doc.getBookmark(bookmarkId)` returns null, no other branch triggers and the issue is silently skipped with no resolution.

Infinite Loop / Performance Issues

`startAutoRefresh` creates a `setInterval` that fires a backend `getAllTags` call every 2 seconds indefinitely — and every time `startAutoRefresh` is called, it creates a new interval with no cleanup mechanism. This compounds over the session. Combined with `handleTagClick` / `renderTags` resetting the sidebar's inner HTML on every tag click, the scroll position resets constantly, making the UI feel unstable and choppy.

Edge Case Gaps

In `collectParagraphBlocksWithTag`, the deduplication logic may accidentally remove two legitimately identical notes filed under the same tag. The `textSignature` variable in `extractHashtagsFromDocument` is computed and returned but never actually used after being returned.

Project Structure

The biggest structural limitation is that `Code.js` bundles tag management, bookmark management, export handling, autocomplete, document utilities, and project management all into one file. Similarly, `Scripts.html` contains all frontend JavaScript for the sidebar in a single file. Some export-related functions also live in `Code.js` instead of `Export.js` where they belong. A testing function (`testTagExtraction`) also sits among production logic with no separation.

Appendix A – Team Review Sign-off

All members of the PaperTrail Development Team have thoroughly reviewed this Administrator Manual Document and agree that it accurately represents the project's goals, scope, and requirements. Each team member confirms that the content and format of this document meet the expectations of both the team and the customer. While minor editorial or stylistic differences may exist, there are no major points of disagreement among the team regarding the material presented herein.

Signatures

Name (Typed)	Signature	Date
Dr. Zihan Wu (Client)	<i>zihan wu</i> zihan wu (Apr 14, 2026 11:43:28 EDT)	4/15/2026
Vasu Patel	<i>VASU patel</i> VASU patel (Apr 14, 2026 11:46:35 EDT)	4/15/2026
Ben Yandell	<i>Benjamin Yandell</i> Benjamin Yandell (Apr 14, 2026 16:33:58 EDT)	4/15/2026
Robert Kulow	<i>Robert Kulow</i> Robert Kulow (Apr 14, 2026 10:42:05 EDT)	4/15/2026
Arius Ahmad	<i>Arius Ahmad</i> Arius Ahmad (Apr 14, 2026 12:07:34 EDT)	4/15/2026
Anthony Veilleux	<i>Anthony Veilleux</i> Anthony Veilleux (Apr 14, 2026 16:14:02 EDT)	4/15/2026

Appendix B – Document Contributions

The following table identifies each team member's contributions to the creation of this Administrator Manual Document. Each member has contributed to writing, editing, and reviewing the document. The percentages represent an estimate of each member's contribution to the total effort.

Team Member	Contributions	Percentage of Work
Vasu Patel	Worked on section 4	20%
Ben Yandell	Finished section 3	20%
Robert Kulow	Completed section 1 and appendix. Managed Signatures and Certificates.	20%
Arius Ahmad	Completed certain parts of section 3	20%
Anthony Veilleux	Completed 2.1,3.1,3.3,4.1	20%

PaperTrial AM

Final Audit Report

2026-04-14

Created:	2026-04-14
By:	Robert Kulow (robert.kulow@maine.edu)
Status:	Signed
Transaction ID:	CBJCHBCAABAAthHVNaeUxsMth7TO9A7lgOE7Paid4XkJ

"PaperTrial AM" History

-  Document created by Robert Kulow (robert.kulow@maine.edu)
2026-04-14 - 2:41:03 PM GMT
-  Document emailed to zihan.wu@maine.edu for signature
2026-04-14 - 2:41:18 PM GMT
-  Document emailed to vasu.patel@maine.edu for signature
2026-04-14 - 2:41:18 PM GMT
-  Document emailed to benjamin.yandell@maine.edu for signature
2026-04-14 - 2:41:18 PM GMT
-  Document emailed to Robert Kulow (robert.kulow@maine.edu) for signature
2026-04-14 - 2:41:19 PM GMT
-  Document emailed to Arius Ahmad (arius.ahmad@maine.edu) for signature
2026-04-14 - 2:41:19 PM GMT
-  Document emailed to anthony.veilleux@maine.edu for signature
2026-04-14 - 2:41:19 PM GMT
-  Document e-signed by Robert Kulow (robert.kulow@maine.edu)
Signature Date: 2026-04-14 - 2:42:05 PM GMT - Time Source: server
-  Email viewed by zihan.wu@maine.edu
2026-04-14 - 3:42:05 PM GMT
-  Signer zihan.wu@maine.edu entered name at signing as zihan wu
2026-04-14 - 3:43:26 PM GMT
-  Document e-signed by zihan wu (zihan.wu@maine.edu)
Signature Date: 2026-04-14 - 3:43:28 PM GMT - Time Source: server

 Email viewed by vasu.patel@maine.edu

2026-04-14 - 3:45:52 PM GMT

 Signer vasu.patel@maine.edu entered name at signing as VASU patel

2026-04-14 - 3:46:33 PM GMT

 Document e-signed by VASU patel (vasu.patel@maine.edu)

Signature Date: 2026-04-14 - 3:46:35 PM GMT - Time Source: server

 Email viewed by Arius Ahmad (arius.ahmad@maine.edu)

2026-04-14 - 4:06:40 PM GMT

 Document e-signed by Arius Ahmad (arius.ahmad@maine.edu)

Signature Date: 2026-04-14 - 4:07:34 PM GMT - Time Source: server

 Email viewed by anthony.veilleux@maine.edu

2026-04-14 - 8:13:41 PM GMT

 Signer anthony.veilleux@maine.edu entered name at signing as Anthony Veilleux

2026-04-14 - 8:14:00 PM GMT

 Document e-signed by Anthony Veilleux (anthony.veilleux@maine.edu)

Signature Date: 2026-04-14 - 8:14:02 PM GMT - Time Source: server

 Email viewed by benjamin.yandell@maine.edu

2026-04-14 - 8:33:33 PM GMT

 Signer benjamin.yandell@maine.edu entered name at signing as Benjamin Yandell

2026-04-14 - 8:33:56 PM GMT

 Document e-signed by Benjamin Yandell (benjamin.yandell@maine.edu)

Signature Date: 2026-04-14 - 8:33:58 PM GMT - Time Source: server

 Agreement completed.

2026-04-14 - 8:33:58 PM GMT